

Práctica: instalación de un servidor LAMP

En esta práctica he instalado y configurado una pila **LAMP** en un servidor Debian. LAMP está formada por Linux, Apache, MariaDB/MySQL y PHP. También he comprobado el funcionamiento del servidor web, he creado una base de datos, he configurado el acceso mediante el nombre `sergiomesamejias.com` y he revisado los logs de Apache.

1. Preparación del sistema

Primero actualicé los repositorios y paquetes de mi servidor Debian:

```
sudo apt update sudo apt upgrade -y
```

```
sergio@app-web:~$ sudo apt update && sudo apt upgrade
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Hit:3 https://deb.debian.org/debian trixie InRelease
Hit:4 https://deb.debian.org/debian trixie-updates InRelease
Hit:5 https://deb.debian.org/debian trixie-backports InRelease
Hit:6 https://deb.debian.org/debian-security trixie-security InRelease
10 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  bind9-host  liblzma5      libpng16-16t64  linux-image-amd64  openssl-provider-legacy
  bind9-libs  libpcre2-8-0  libssl3t64      openssl            xz-utils
Installing dependencies:
  linux-image-6.12.111+deb13-amd64
```

Consulté la dirección IP de mi servidor, ya que la necesitaría más adelante para configurar el acceso desde el equipo cliente:

```
hostname -I
```

```
sergio@app-web:~$ hostname -I
192.168.122.198
sergio@app-web:~$ ipa
```

2. Instalación de MariaDB

MariaDB será el gestor de bases de datos utilizado en mi pila LAMP.

Instalé el servidor MariaDB con el siguiente comando:

```
sudo apt install mariadb-server -y
```

```

sergio@app-web:~$ sudo apt install mariadb-server -y
Installing:
  mariadb-server

Installing dependencies:
  galera-4          libhttp-date-perl      liburi-perl
  gawk              libhttp-message-perl  lsof
  libcgi-fast-perl  libio-compress-brotli-perl mariadb-client
  libcgi-pm-perl    libio-html-perl       mariadb-client-core
  libclone-perl     liblsof0              mariadb-common
  libconfig-inifiles-perl liblwp-mediatypes-perl mariadb-plugin-provider-bzip2
  libdbd-mariadb-perl liblz02-2             mariadb-plugin-provider-lz4
  libdbi-perl       libmariadb3           mariadb-plugin-provider-lzma
  libencode-locale-perl libmpfr6              mariadb-plugin-provider-lzo
  libfcgi-bin       libncurses6           mariadb-plugin-provider-snappy
  libfcgi-perl      libpcre2-posix3       mariadb-server-core
  libfcgi0t64       libperl5.40           mysql-common
  libgdbm-compat4t64 libsigsegv2           perl
  libhtml-parser-perl libsnappy1v5          perl-modules-5.40
  libhtml-tagset-perl libterm-readkey-perl  pv
  libhtml-template-perl libtimedate-perl      rsync

Suggested packages:
  gawk-doc          libmime-base32-perl  | libterm-readline-perl-perl
  libmldbm-perl     libregex-ipv6-perl  make
  libnet-daemon-perl libwww-perl          libtap-harness-archive-perl
  libsql-statement-perl mailx                  doc-base
  libdata-dump-perl mariadb-test         python3-braceexpand
  libipc-sharedcache-perl perl-doc
  libbusiness-isbn-perl libterm-readline-gnu-perl

Summary:
  Upgrading: 0, Installing: 49, Removing: 0, Not Upgrading: 0
  Download size: 29.4 MB
  Space needed: 255 MB / 18.9 GB available

```

Una vez instalado, comprobé que el servicio estaba funcionando correctamente:

```
sudo systemctl status mariadb
```

```

sergio@app-web:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-08 11:05:46 UTC; 9s ago
  Invocation: b039b124610c4e61b4c62bc8ba3de365

```

También activé el servicio para que MariaDB se inicie automáticamente cada vez que arranque el servidor:

```
sudo systemctl enable --now mariadb
```

```

sergio@app-web:~$ sudo systemctl enable --now mariadb
> ^C
sergio@app-web:~$ sudo systemctl enable --now mariadb
Synchronizing state of mariadb.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mariadb
sergio@app-web:~$ sudo systemctl is-enabled mariadb
enabled

```

Configuración segura de MariaDB

A continuación ejecuté el script de seguridad de MariaDB:

```
sudo mariadb-secure-installation
```

Durante la configuración realicé las siguientes acciones:

- Establecí o configuré la contraseña del usuario administrador de MariaDB.
- Eliminé los usuarios anónimos.
- Impedí el acceso remoto del usuario `root`.
- Eliminé la base de datos de prueba.
- Recargué los privilegios de MariaDB.

Estas opciones permiten dejar la instalación de MariaDB más segura.

Creación de base de datos y usuario

Después accedí a MariaDB como administrador:

```
sudo mariadb
```

Dentro de la consola de MariaDB creé una base de datos llamada `lampdb`:

```
CREATE DATABASE lampdb;
```

```
MariaDB [(none)]> CREATE DATABASE lampdb;  
Query OK, 1 row affected (0.001 sec)
```

Creé un usuario llamado `lampuser`, con acceso desde el propio servidor:

```
CREATE USER 'lampuser'@'localhost' IDENTIFIED BY 'ContraseñaSegura123!';
```

```
MariaDB [(none)]> CREATE USER 'sergio'@'localhost' IDENTIFIED BY 'sergio1223!';  
Query OK, 0 rows affected (0.035 sec)
```

Asigné todos los permisos sobre la base de datos `lampdb` a dicho usuario:

```
GRANT ALL PRIVILEGES ON lampdb.* TO 'lampuser'@'localhost';
```

```
MariaDB [(none)]> GRANT ALL PRIVILEGES ON lampdb.* TO 'sergio'@'localhost';  
Query OK, 0 rows affected (0.036 sec)
```

Actualicé los privilegios:

```
FLUSH PRIVILEGES;
```

```
MariaDB [(none)]> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.001 sec)
```

Y para salir usamos:

```
EXIT;
```

```
MariaDB [(none)]> EXIT;  
Bye  
sergio@app-web:~$
```

3. Instalación de Apache

A continuación instalé Apache, que será el servidor web encargado de servir las páginas de mi servidor LAMP.

Ejecuté:

```
sudo apt install apache2 -y
```

```
bye  
sergio@app-web:~$ sudo apt install apache2 -y  
Installing:  
  apache2  
  
Installing dependencies:  
  apache2-bin  apache2-utils  libaprutil1-dbd-sqlite3  libaprutil1t64  liblua5.4-0  
  apache2-data  libapr1t64  libaprutil1-ldap  libjansson4  ssl-cert  
  
Suggested packages:  
  apache2-doc  apache2-suexec-pristine | apache2-suexec-custom  ufw  www-browser
```

Después comprobé el estado del servicio:

```
sudo systemctl status apache2
```

```
sergio@app-web:~$ sudo systemctl status apache2  
● apache2.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)  
   Active: active (running) since Thu 2026-10-08 11:13:01 UTC; 17s ago  
 Invocation: 80ee5dbb23d84ee4937a5557b954572c  
    Docs: https://httpd.apache.org/docs/2.4/
```

También configuré Apache para que se inicie automáticamente cuando arranque el sistema:

```
sudo systemctl enable apache2
```

```
sergio@app-web:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv
ll.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
sergio@app-web:~$
```

Reinicié Apache:

```
sudo systemctl restart apache2
```

Por último, comprobé desde el propio servidor que Apache respondía correctamente:

```
curl -I http://localhost
```

```
sergio@app-web:~$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Thu, 08 Oct 2026 11:14:54 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 08 Oct 2026 11:12:58 GMT
ETag: "29cf-65d5250714985"
Accept-Ranges: bytes
Content-Length: 10703
Vary: Accept-Encoding
Content-Type: text/html
```

4. Instalación de PHP

Una vez instalado Apache, instalé PHP y los módulos necesarios para que Apache pueda interpretar código PHP y PHP pueda conectarse con MariaDB.

Ejecuté el siguiente comando:

```
sudo apt install apache2 libapache2-mod-php php php-mysql -y
```

```
sergio@app-web:~$ sudo apt install apache2 libapache2-mod-php php php-mysql -y
apache2 is already the newest version (2.4.68-1~deb13u1).
Installing:
  libapache2-mod-php  php  php-mysql

Installing dependencies:
  libapache2-mod-php8.4  php-common  php8.4-cli  php8.4-mysql  php8.4-readline
  libargon2-1            php8.4      php8.4-common  php8.4-opcache

Suggested packages:
  php-pear
```

También instalé algunos módulos adicionales de PHP que pueden ser necesarios en aplicaciones web:

```
sudo apt install php-cli php-curl php-mbstring php-xml php-zip -y
```

```
sergio@app-web:~$ sudo apt install php-cli php-curl php-mbstring php-xml php-zip -y
Installing:
  php-cli  php-curl  php-mbstring  php-xml  php-zip

Installing dependencies:
  libgcrypt20      libgpg-error0  libxslt1.1  php8.4-curl  php8.4-xml
  libgpg-error-l10n  libonig5      libzip5     php8.4-mbstring  php8.4-zip

Suggested packages:
  rng-tools
```

Después comprobé la versión instalada de PHP:

```
php -v
```

```
sergio@app-web:~$ php -v
PHP 8.4.26 (cli) (built: Sep 24 2026 17:16:18) (NTS)
Copyright (c) The PHP Group
Built by Debian
Zend Engine v4.4.26, Copyright (c) Zend Technologies
    with Zend OPcache v8.4.26, Copyright (c), by Zend Technologies
sergio@app-web:~$
```

También revisé los módulos cargados:

```
php -m
```

```
sergio@app-web:~$ php -m
```

```
[PHP Modules]
```

```
calendar
```

```
Core
```

```
ctype
```

```
curl
```

```
date
```

```
dom
```

```
exif
```

```
FFI
```

```
fileinfo
```

```
filter
```

```
ftp
```

```
gettext
```

```
hash
```

```
iconv
```

```
json
```

```
libxml
```

```
mbstring
```

```
mysqli
```

```
mysqlnd
```

```
openssl
```

```
pcntl
```

```
pcre
```

```
PDO
```

```
pdo_mysql
```

```
Phar
```

```
posix
```

```
random
```

```
readline
```

```
Reflection
```

```
session
```

```
shmop
```

```
SimpleXML
```

```
sockets
```

```
sodium
```

```
SPL
```

```
standard
```

```
sysvmsg
```

```
sysvsem
```

```
sysvshm
```

```
tokenizer
```

```
xml
```

```
xmlreader
```

```
xmlwriter
```

```
xsl
```

```
Zend OPcache
```

```
zip
```

```
zlib
```

```
[Zend Modules]
```

```
Zend OPcache
```

Para verificar específicamente que PHP podía trabajar con MariaDB/MySQL, ejecuté:

```
php -m | grep -i mysql
```

```
sergio@app-web:~$ php -m | grep -i mysql
mysqli
mysqlnd
pdo_mysql
```

Finalmente reinicié Apache para que cargara correctamente el módulo de PHP:

```
bash
```

```
sudo systemctl restart apache2
```

5. Comprobación de PHP

Para comprobar que Apache podía ejecutar código PHP, creé el archivo `info.php` dentro del directorio principal del servidor web, que por defecto es `/var/www/html`.

Creé el fichero:

```
sudo nano /var/www/html/info.php
```

Dentro escribí el siguiente código:

```
<?php phpinfo(); ?>
```

```
sergio@app-web:~$ sudo nano /var/www/html/info.php
sergio@app-web:~$ sudo cat /var/www/html/info.php
<?php phpinfo(); ?>
sergio@app-web:~$
```

Guardé el archivo y le asigné como propietario el usuario y grupo utilizados por Apache:

```
sudo chown www-data:www-data /var/www/html/info.php
```

```
sergio@app-web:~$ sudo chown www-data:www-data /var/www/html/info.php
sergio@app-web:~$
```

Después abrí la página desde el navegador utilizando la IP del servidor:

```
http://192.168.122.198/info.php
```

PHP Version 8.4.26



System	Linux app-web 6.12.107+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.107-1 (2026-08-29) x86_64
Build Date	Sep 24 2026 17:16:18
Build System	Linux
Build Provider	Debian
Server API	Apache 2 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/8.4/apache2
Loaded Configuration File	/etc/php/8.4/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/8.4/apache2/conf.d
Additional .ini files parsed	/etc/php/8.4/apache2/conf.d/10-mysqld.ini, /etc/php/8.4/apache2/conf.d/10-opcache.ini, /etc/php/8.4/apache2/conf.d/10-pdo.ini, /etc/php/8.4/apache2/conf.d/15-xml.ini, /etc/php/8.4/apache2/conf.d/20-calendar.ini, /etc/php/8.4/apache2/conf.d/20-ctype.ini, /etc/php/8.4/apache2/conf.d/20-curl.ini, /etc/php/8.4/apache2/conf.d/20-dom.ini, /etc/php/8.4/apache2/conf.d/20-exif.ini, /etc/php/8.4/apache2/conf.d/20-ffi.ini, /etc/php/8.4/apache2/conf.d/20-fileinfo.ini, /etc/php/8.4/apache2/conf.d/20-ftp.ini, /etc/php/8.4/apache2/conf.d/20-gettext.ini, /etc/php/8.4/apache2/conf.d/20-iconv.ini, /etc/php/8.4/apache2/conf.d/20-mbstring.ini, /etc/php/8.4/apache2/conf.d/20-mysqli.ini, /etc/php/8.4/apache2/conf.d/20-pdo_mysql.ini, /etc/php/8.4/apache2/conf.d/20-phar.ini, /etc/php/8.4/apache2/conf.d/20-posix.ini, /etc/php/8.4/apache2/conf.d/20-readline.ini, /etc/php/8.4/apache2/conf.d/20-shmop.ini, /etc/php/8.4/apache2/conf.d/20-simplexml.ini, /etc/php/8.4/apache2/conf.d/20-sockets.ini, /etc/php/8.4/apache2/conf.d/20-sysvmsg.ini, /etc/php/8.4/apache2/conf.d/20-sysvsem.ini, /etc/php/8.4/apache2/conf.d/20-sysvshm.ini, /etc/php/8.4/apache2/conf.d/20-tokenizer.ini, /etc/php/8.4/apache2/conf.d/20-xmlreader.ini, /etc/php/8.4/apache2/conf.d/20-xmlwriter.ini, /etc/php/8.4/apache2/conf.d/20-xsl.ini, /etc/php/8.4/apache2/conf.d/20-zip.ini
PHP API	20240924
PHP Extension	20240924
Zend Extension	420240924
Zend Extension Build	API420240924,NTS
PHP Extension Build	API20240924,NTS
PHP Integer Size	64 bits
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
Zend Max Execution Timers	disabled
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar, zip
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2, tlsv1.3
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, convert.*, consumed, dechunk, convert.iconv.*

La página mostró la información de PHP, incluyendo la versión instalada, el servidor Apache, los módulos cargados y los datos de configuración.

6. Creación de la página principal

Para mostrar una página propia desde mi servidor, creé un archivo llamado `index.php` :

```
sudo nano /var/www/html/index.php
```

Introduje el siguiente contenido:

```
sergio@app-web:~$ sudo cat /var/www/html/index.php
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Servidor LAMP</title>
</head>
<body>
  <h1>Servidor LAMP funcionando</h1>
  <p>Apache está funcionando correctamente.</p>
  <p>PHP está funcionando correctamente.</p>
</body>
</html>
sergio@app-web:~$
```

Guardé el archivo y le asigné los permisos correctos:

```
sudo chown www-data:www-data /var/www/html/index.php
```

```
sergio@app-web:~$ sudo chown www-data:www-data /var/www/html/index.php
```

Por último, comprobé el resultado desde el propio servidor:

```
curl http://localhost/index.php
```

```
sergio@app-web:~$ curl http://localhost/index.php
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Servidor LAMP</title>
</head>
<body>
  <h1>Servidor LAMP funcionando</h1>
  <p>Apache está funcionando correctamente.</p>
  <p>PHP está funcionando correctamente.</p>
</body>
</html>
```

7. Configuración del dominio local

Para acceder a mi web mediante el nombre `sergiomesamejias.com`, configuré una resolución estática utilizando el archivo `/etc/hosts`.

En mi caso, la IP utilizada es:

```
192.168.122.198
```

Después, desde el equipo cliente —el ordenador desde el que iba a abrir la página web— edité el fichero `/etc/hosts` :

```
sudo nano /etc/hosts
```

Añadí la siguiente línea:

```
192.168.121.10 sergiomesamejias.com www.sergiomesamejias.com
```

El contenido relevante del fichero quedó así:

```
127.0.0.1      localhost 192.168.121.10  sergiomesamejias.com
```

```
www.sergiomesamejias.com
```

```
sergio@Sergio-PC:~$ sudo nano /etc/hosts
sergio@Sergio-PC:~$ cat /etc/hosts
127.0.0.1      localhost
127.0.1.1      Sergio-PC.gonzalonazareno.org Sergio-PC

# The following lines are desirable for IPv6 capable hosts

::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

172.22.200.150 apache.sergio.org      apache
192.168.122.101 sergiotarea1.gonzalonazareno.org sergiotarea1
192.168.122.41  bd.sergio.serv  oracle
192.168.122.55  oracle.sergio.bd      oracle
192.168.122.221 router.sergio.org      router
192.168.100.20  nas.sergio.org  nas
192.168.100.30  web.sergio.org  web
192.168.122.221 data.sergio.org
192.168.122.198 sergiomesamejias.com www.sergiomesamejias.com
sergio@Sergio-PC:~$
```

De esta forma, mi ordenador asocia el nombre `sergiomesamejias.com` con la dirección IP privada del servidor, sin necesidad de tener un servidor DNS propio.

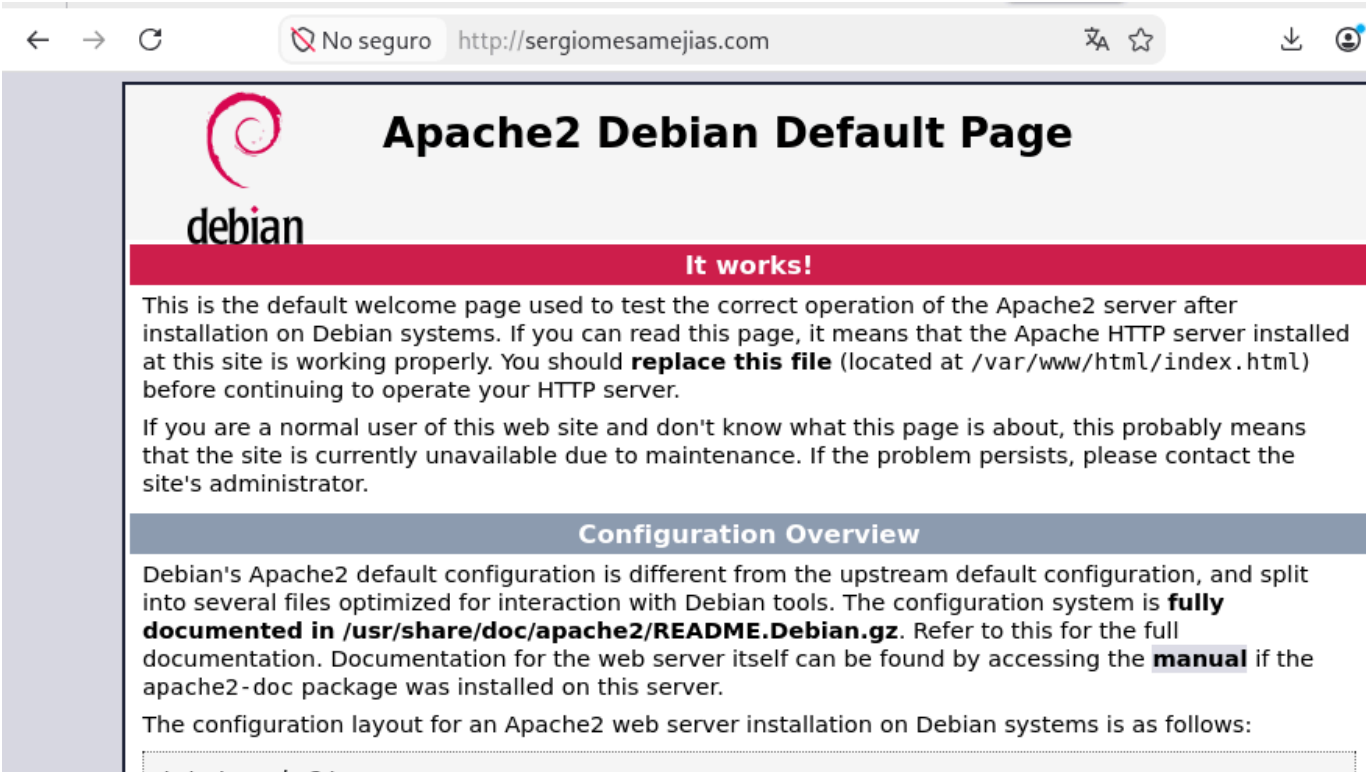
Comprobé que el nombre resolvía correctamente:

```
ping -c 4 sergiomesamejias.com
```

```
sergio@Sergio-PC:~$ ping -c 4 sergiomesamejias.com
PING sergiomesamejias.com (192.168.122.198) 56(84) bytes of data.
64 bytes from sergiomesamejias.com (192.168.122.198): icmp_seq=1 ttl=64 time=0.557 ms
64 bytes from sergiomesamejias.com (192.168.122.198): icmp_seq=2 ttl=64 time=0.521 ms
^C
--- sergiomesamejias.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1024ms
rtt min/avg/max/mdev = 0.521/0.539/0.557/0.018 ms
```

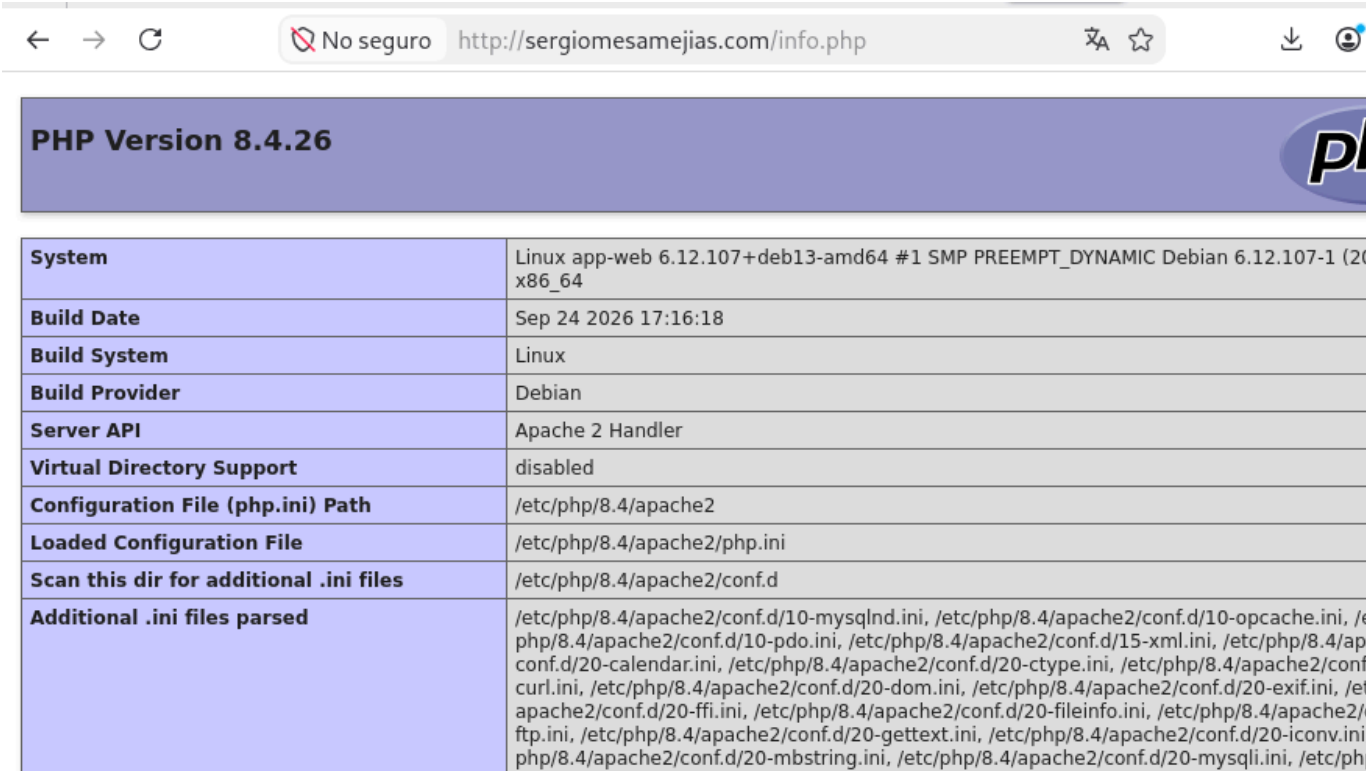
Finalmente accedí desde el navegador a:

http://sergiomesamejias.com/



También pude acceder a la página de PHP mediante:

http://sergiomesamejias.com/info.php



La resolución estática mediante `/etc/hosts` permite asociar manualmente una IP con un nombre de dominio cuando no se utiliza un servidor DNS. [Parte-1-Introduccion-a-PHP.pdf](#)

8. Prueba de conexión entre PHP y MariaDB

Para comprobar que PHP podía conectarse correctamente a MariaDB, creé un archivo llamado `bd.php` :

```
sudo nano /var/www/html/bd.php
```

Introduce el siguiente código, utilizando el usuario y la base de datos creados anteriormente:

```
sergio@app-web:~$ sudo nano /var/www/html/bd.php
sergio@app-web:~$ sudo cat /var/www/html/bd.php
<?php

$conexion = new mysqli(
    "localhost",
    "sergio",
    "sergio1223!",
    "lampdb"
);

if ($conexion->connect_error) {
    die("Error de conexión: " . $conexion->connect_error);
}

echo "Conexión correcta entre PHP y MariaDB";
?>
sergio@app-web:~$
```

Después establecí el propietario adecuado para Apache:

```
sudo chown www-data:www-data /var/www/html/bd.php
```

Abrí la página desde el navegador:

```
http://sergiomesamejias.com/bd.php
```



9. Consulta de logs de Apache

Por último, revisé los logs de Apache para comprobar los accesos, los posibles errores y los mensajes del servicio.

Los archivos principales de Apache en Debian son:

text

```
/var/log/apache2/access.log /var/log/apache2/error.log
```

Log de accesos

Mostré las últimas 20 líneas del log de accesos:

bash

```
sudo tail -n 20 /var/log/apache2/access.log
```

También lo visualicé en tiempo real:

bash

```
sudo tail -f /var/log/apache2/access.log
```

Mientras el comando estaba activo, abrí la página `http://sergiomesamejias.com/` desde el navegador. De esta forma aparecieron nuevas peticiones registradas en el log.

Una línea típica fue similar a esta:

text

```
192.168.121.20 - - [08/Oct/2026:13:20:10 +0200] "GET / HTTP/1.1" 200 2456
```

En ella se puede ver la IP del cliente, la fecha y hora, el recurso solicitado y el código HTTP de respuesta.

Para salir de la visualización en tiempo real pulsé:

text

```
ctrl + c
```

Log de errores

Después consulté el archivo de errores:

bash

```
sudo tail -n 20 /var/log/apache2/error.log
```

También podía verlo en tiempo real con:

```
bash
```

```
sudo tail -f /var/log/apache2/error.log
```

Si no aparecían errores recientes, significaba que Apache estaba funcionando correctamente.

Logs del servicio Apache

Finalmente consulté los mensajes del servicio Apache mediante `journalctl`:

```
bash
```

```
sudo journalctl -u apache2
```

Para mostrar solamente las últimas líneas utilicé:

```
bash
```

```
sudo journalctl -u apache2 -n 30 --no-pager
```

También consulté los mensajes de Apache desde el último arranque:

```
bash
```

```
sudo journalctl -u apache2 -b --no-pager
```

Los logs de Apache y los mensajes de `journalctl -u apache2` son los métodos indicados en el material de clase para localizar accesos, errores y problemas en el servicio web. [Parte-1-Introduccion-a-PHP.pdf](#)

10. Comprobación final

Para terminar la práctica, comprobé que todos los servicios y componentes de la pila LAMP estaban funcionando correctamente.

Comprobé el estado de Apache:

```
bash
```

```
sudo systemctl status apache2
```

Comprobé el estado de MariaDB:

```
bash
```

```
sudo systemctl status mariadb
```

Comprobé la versión de PHP:

```
bash
```

```
php -v
```

Comprobé la configuración de Apache:

```
bash
```

```
sudo apache2ctl configtest
```

La respuesta fue:

```
text
```

```
Syntax OK
```

Comprobé la página desde el nombre configurado:

```
bash
```

```
curl -I http://sergiomesamejias.com/
```

La respuesta correcta fue similar a esta:

```
text
```

```
HTTP/1.1 200 OK
```

Por último, revisé los puertos de Apache y MariaDB:

```
bash
```

```
sudo ss -tulpn | grep -E ' :80|:3306 '
```

Con este comando comprobé que Apache estaba utilizando el puerto 80 y que MariaDB estaba disponible en el puerto 3306, normalmente limitado al propio servidor.

Conclusión

En esta práctica he instalado y configurado correctamente una pila LAMP en Debian. He instalado MariaDB como servidor de bases de datos, Apache como servidor web y PHP junto con los módulos necesarios para conectarlo con MariaDB.

Además, he creado una base de datos llamada `lampdb` y un usuario llamado `lampuser`, he comprobado que Apache interpreta correctamente código PHP mediante `info.php` y he realizado una prueba de conexión entre PHP y MariaDB.

Por último, he configurado el nombre `sergiomesamejias.com` mediante el archivo `/etc/hosts`, por lo que he podido acceder al servidor web desde el equipo cliente usando un nombre en lugar de la dirección IP. También he consultado los logs de acceso, errores y del servicio Apache para comprobar y diagnosticar el funcionamiento del servidor.